

Artifact Appendix: Versatile yet Efficient Network Traffic Analysis: Offloading Network Foundation Model to SmartNIC

Badges requested: Available and Functional (Reproduced is not requested)

Artifact Overview

This artifact implements Nepco, a network foundation model designed for early-stage encrypted-traffic analysis and deployment on an NVIDIA BlueField-3 SmartNIC. The release contains the model and training code, configuration and vocabulary, a pretrained checkpoint, packet-capture preprocessing, fine-tuning, host-side latency measurement, ONNX export and quantization, BlueField-3 sender/receiver programs, and adversarial-robustness evaluation. The requested evaluation establishes that the released components are complete, documented, and executable independently of the authors' development machines. It is not a request to reproduce every quantitative result or baseline in the paper.

Description and Requirements

How to Access

The archival artifact is available at [ARTIFACT DOI/URL TO BE INSERTED]. The archive should be cited by its DOI and verified against the SHA-256 checksum published with the release. After extraction, all commands below are run from the artifact root.

Artifact Layout

- configs/ - Nepco architecture configuration.
- uer/ - Nepco model, training, data, and utility code.
- vocab/ - Four-hex-character vocabulary.
- models/ - Released pretrained checkpoint and tokenizer metadata.
- scripts/1_*/ - PCAP-to-corpus and labeled-data generation.
- scripts/2_*/ - Pre-training and fine-tuning.
- scripts/3_*/ - Host CPU/GPU latency measurement.
- scripts/4_*/ - ONNX conversion and BlueField-3 deployment.
- scripts/5_*/ - Adversarial-robustness workflow.

Hardware Dependencies

The software workflow runs on a recent x86-64 Linux workstation. We recommend at least 8 CPU cores, 32 GB RAM, and 20 GB free disk space in addition to the selected public datasets and generated outputs. A CUDA-capable NVIDIA GPU with at least 16 GB memory is recommended for fine-tuning and the GPU latency workflow; the host CPU latency workflow does not require a GPU. Full pre-training (100,000 steps) is a long-running experiment and requires substantially more compute.

The SmartNIC workflow requires two Linux hosts connected through NVIDIA BlueField-3/ConnectX-7 interfaces, with DPDK-compatible ports and one BlueField-3 operating as the receiver. The paper testbed used two directly connected servers,

each with a 5.4 GHz Intel Core i7-14700 CPU, 64 GB DRAM, a BlueField-3 SmartNIC, and a 200 Gbps link. Commodity hosts can evaluate preprocessing, model execution, and ONNX export, but cannot validate the SmartNIC deployment without equivalent hardware.

Software Dependencies

Use a recent Linux distribution with Python 3.10 or later. Python dependencies are listed in requirements.txt: PyTorch, NumPy, scikit-learn, Scapy, tqdm, ONNX, and ONNX Runtime. The paper software prototype used PyTorch 2.3.0 with CUDA 12.4. The SmartNIC workflow additionally requires a BlueField-3 SDK/DOCA environment, DPDK, a C++ compiler, GNU Make, root privileges for device binding and execution, and an ONNX Runtime C/C++ installation on the BlueField Arm system. Network access is required only to obtain the artifact, Python packages, and public datasets.

Datasets

The artifact accepts PCAP/PCAPNG inputs and does not redistribute third-party traffic captures. The paper pre-trained on Bottlenet-HTTPS and ISCX-SlowDoS, and fine-tuned on USTC-TFC, ISCX-VPN-App, ISCX-VPN-Service, DataCon, CIC-EVSE, and CIC-IoMT. Reviewers may use any legally obtained labeled PCAP subset for the functional workflow. Dataset download, licensing, and storage are external to the artifact.

Installation and Configuration

Create an isolated Python environment and install the dependencies:

```
python3 -m venv .venv
source .venv/bin/activate
python3 -m pip install -r requirements.txt
python3 -c "import torch,scapy,onnx; print('OK!')"
```

Confirm that models/nepco_pretrained_model.bin, configs/nepco_config.json, and vocab/hex_vocab.txt are present. GPU users should additionally verify torch.cuda.is_available(). Detailed flags and output paths are documented in the top-level README.md.

Functional Scope and Claims

- F1--Processing. The release converts raw packet captures into Nepco's address-sanitized, four-hex-character token representation and creates deterministic 8:1:1 labeled splits.
- F2--Model execution. The released checkpoint, architecture, and vocabulary instantiate Nepco; the scripts support fine-tuning and host-side CPU/GPU inference on the first five packets of each flow.
- F3--Deployment path. A fine-tuned model can be exported to ONNX, preprocessed, statically quantized, loaded by the BlueField-3 receiver, and exercised by the paired sender.
- F4--Robustness path. The evaluation constructs equal-budget Top-X and Random-X byte perturbations from gradient attribution and evaluates both raw and attack-aware models.

These functional checks support the paper's system design and implementation claims. They do not by themselves reproduce the multi-dataset accuracy table, the full baseline comparison, the 20-seed confidence intervals, or the complete SmartNIC load study.

Evaluation Workflow

E1: Data Processing

Effort: about 5 human-minutes plus dataset-dependent compute time. Place a small labeled PCAP subset in one directory per class and run:

```
PCAP_DIR=/path/to/labeled_pcaps CLASS_NUM=N \
bash scripts/1_input_data_processing/\
run_finetune_data_generation.sh
```

Successful execution creates `train_dataset.tsv`, `valid_dataset.tsv`, `test_dataset.tsv`, and `no_label_test_dataset.tsv` under `outputs/finetune_data/datasets/`. Inspect several rows and confirm that samples are token sequences with integer labels. E1 evaluates F1.

E2: Fine-tuning and Host Inference

Effort: about 5 human-minutes; compute time depends on the subset and GPU. With the E1 files, run:

```
TRAIN_PATH=outputs/finetune_data/datasets/train_dataset.tsv \
DEV_PATH=outputs/finetune_data/datasets/valid_dataset.tsv \
TEST_PATH=outputs/finetune_data/datasets/test_dataset.tsv \
LABELS_NUM=N EPOCHS_NUM=1 BATCH_SIZE=8 \
bash scripts/2_model_train/model_finetune/run_finetune.sh
```

The smoke configuration uses one epoch; the paper configuration uses ten. Successful execution writes `outputs/model_finetune/models/finetuned_model.bin` and `prf/prf.csv`. Next run the CPU latency wrapper with that checkpoint and test file. It should load the model, complete inference, report per-sample latency over several batch sizes, and update the classification summary. E2 evaluates F2.

E3: ONNX Export

Effort: about 5 human-minutes plus several compute-minutes. Run:

```
FINETUNED_MODEL_PATH=outputs/model_finetune/models/\
finetuned_model.bin \
TRAIN_PATH=outputs/finetune_data/datasets/train_dataset.tsv \
LABELS_NUM=N \
bash scripts/4_latency_measure_SmartNIC/\
1_onnx_conversion/run_onnx_export.sh
```

Successful execution creates `output.onnx`, `output.preprocessed.onnx`, and `output.static_int8.onnx` under `outputs/onnx/`. Load each generated graph with ONNX's checker. E3 validates the portable model-conversion part of F3.

E4: BlueField-3 Deployment (Hardware-Specific)

Effort: 20-40 human-minutes after the BlueField/DPDK environment is operational. On the receiver, build with explicit `MODEL_PATH`, `VOCAB_PATH`, `LABELS_NUM`, and `ONNX_ROOT`; build the sender on the peer host. Bind the intended interfaces to DPDK, verify PCI addresses, start the receiver, and then start the sender. The sender should print per-second Mpps/Gbps. The receiver should print and log queue-wait, input-construction, model-inference, and total analysis latency, and should create `log.txt` and `flow_tuples.txt`. E4 completes F3. Exact values depend on hardware and offered load; paper-number reproduction is outside the requested badge scope.

E5: Adversarial-Robustness Smoke

Effort: about 5 human-minutes plus GPU-dependent compute time. Invoke `run_adversarial_robustness.sh` with the E1 train/validation/test files, `LABELS_NUM=N`, one attack seed, one perturbation ratio, and one fine-tuning epoch. The run should create an `adv_summary_AttackSeed_1.csv` containing clean, Top-X, Random-X, and attack-aware results. E5 evaluates F4; reproducing the complete robustness figure requires the paper's full dataset, all ratios, and all model baselines.

Expected Results and Interpretation

The functional criterion is successful end-to-end execution with the documented files and metrics, not numerical agreement with every paper table. E1 should produce valid deterministic splits; E2 should train and infer without shape or checkpoint errors; E3 should produce valid ONNX graphs; E4 should receive flows, execute inference, and log latency on BlueField-3; and E5 should produce matched-budget perturbation results. Classification scores vary with the selected data subset, split seed, training duration, and hardware. Timing values are not comparable across different CPUs, GPUs, SmartNICs, or traffic generators.

Customization and Notes

Reviewers may change the dataset, number of classes, split seed, batch size, learning rate, or perturbation ratio through the documented environment variables and command-line arguments. Keep `token_nibbles=4`, `seq_length=128`, and `max_packets_per_flow=5` when checking the paper configuration. Address and transport-port fields are zeroed in the model input to prevent shortcut learning; the original five-tuple remains available to the SmartNIC flow table. Record all deviations from the paper settings when reporting results.